CSCI-GA 3033: Building LLM Reasoners, Spring 2026

Systems and Parallelism

Aditeya Baral
N19186654 — ab12057

Problem 1: End-to-End Benchmarking

Problem a

Solution

The file can be found at `student/benchmark.py`

Problem b

Model	Forward (ms)	Backward (ms)	Forward + Backward (ms)
small	14.88 ± 0.18	41.21 ± 0.06	55.81 ± 0.12
medium	46.44 ± 0.02	125.52 ± 0.13	170.68 ± 0.12
large	119.35 ± 0.21	287.88 ± 0.30	405.77 ± 0.18
xl	220.72 ± 0.16	543.24 ± 0.56	760.15 ± 0.31
2.7B	281.42 ± 0.26	663.62 ± 0.61	941.69 ± 1.46

Table 1: Forward, backward, and total pass timings (5 warmup steps, 10 measurement steps, context length 512, batch size 4, FP32).

Solution

Forward pass times range from 14.88 ms for the small model to 281.42 ms for the 2.7B model. The backward pass consistently takes roughly 2.4–2.8× longer than the forward pass across all model sizes, which is expected as the backward pass must compute and store gradients for all parameters. Standard deviations are very small across all configurations (well under 1% of the mean), indicating highly stable measurements once the GPU is warmed up.

Problem c

Warmup	Model	Forward (ms)	Backward (ms)	Forward + Backward (ms)
0	small	59.88 ± 142.22	41.42 ± 0.07	65.90 ± 31.04
	medium	90.14 ± 137.08	125.76 ± 0.22	179.16 ± 25.33
	large	163.56 ± 134.89	288.67 ± 0.38	415.10 ± 23.41
	xl	265.41 ± 137.63	544.82 ± 0.71	770.05 ± 20.33
	2.7B	325.90 ± 139.55	659.78 ± 1.14	944.46 ± 20.11
1	small	14.90 ± 0.06	41.41 ± 0.08	56.08 ± 0.27
	medium	46.80 ± 0.27	125.98 ± 0.13	171.27 ± 0.21
	large	120.31 ± 0.40	289.00 ± 0.23	408.08 ± 0.32
	xl	221.78 ± 0.24	545.50 ± 0.54	763.94 ± 0.90
	2.7B	281.58 ± 0.32	661.86 ± 1.26	939.01 ± 0.74
2	small	14.88 ± 0.07	41.33 ± 0.06	55.89 ± 0.08
	medium	46.99 ± 0.06	126.00 ± 0.13	171.48 ± 0.09
	large	120.41 ± 0.27	288.96 ± 0.23	408.07 ± 0.17
	xl	222.03 ± 0.18	545.35 ± 0.38	764.17 ± 0.43
	2.7B	282.00 ± 0.28	662.46 ± 0.80	939.46 ± 1.41

Table 2: Forward, backward, and total pass timings across model sizes with 0, 1, and 2 warmup steps (10 measurement steps, context length 512, batch size 4, FP32). For reference, results with 5 warmup steps are shown in Table 1.

Solution

Without any warmup steps, forward pass times are both inflated and highly variable — for example, the small model shows 59.88 ± 142.22 ms compared to a stable 14.88 ± 0.18 ms with 5 warmup steps. This is because the first CUDA kernel invocations trigger JIT compilation and one-time GPU initialization, which are not representative of steady-state performance. Interestingly, the backward-only timings are already stable at 0 warmup steps: since the backward timer starts only after the forward pass has completed (outside the timer), the forward pass itself acts as an implicit warmup. With just 1 warmup step the forward and total timings recover to near their steady-state values, and by 2 warmup steps results are essentially indistinguishable from the 5-warmup baseline.

Problem 2: Nsight Systems Profiler

Problem a

Solution

At context length 512, the nsys `forward_iteration` NVTX timings closely match the Python `timeit` measurements for medium and larger models: e.g., 46.9 ms vs. 46.4 ms for medium, 120.4 ms vs. 119.4 ms for large, and 280.2 ms vs. 281.4 ms for 2.7B (all within 1%). The small model shows a larger discrepancy (17.3 ms vs. 14.9 ms, +16%) because nsys instrumentation adds a roughly fixed per-kernel overhead that is proportionally more significant when total runtime is short. For large models the overhead is negligible.

Problem b

Solution

For the forward pass, the top kernel by cumulative GPU time varies by model size. For the small model at ctx=128, the dominant GEMM is `cutlass_80_tensorop_s1688gemm_128x128_16x5_nt_align4` (NT layout, corresponding to the linear layer forward pass $Y = XW^\top$ where W is stored as `(out, in)`) at 8.7% of total profile time. For the 2.7B model at ctx=512, the same NT-layout linear forward kernels dominate the forward pass. When profiling the full forward+backward pass, the dominant kernel shifts to TN-layout GEMMs (`tn` suffix), which correspond to weight-gradient computations $\mathrm{d}W = X^\top \mathrm{d}Y$ in the backward pass; for example, `cutlass_80_tensorop_s1688gemm_256x128_16x3_tn_align4` accounts for 16.8% of total time in the 2.7B ctx=1024 training profile across 538 invocations. The layout conventions are: NT = linear forward ($Y = XW^\top$); TN = weight gradient ($\mathrm{d}W = X^\top \mathrm{d}Y$); NN = input gradient ($\mathrm{d}X = \mathrm{d}Y\, W$).

Problem c

Solution

Beyond matrix multiplications, the 2.7B ctx1024 profile reveals several other significant kernel categories. Softmax-related kernels collectively account for a large fraction: `exp_kernel` (7.1%, 270 instances), `reduce_kernel` with `MaxOps` (3.7%, row-wise maximum for numerically stable softmax), and `reduce_kernel` with `func_wrapped` (3.7%, row-wise sum for normalization). A `BUnaryFunctor` kernel (7.2%, pointwise negation/scaling) appears in attention pre-processing and RoPE. `BinaryFunctor` kernels (6.5%) handle residual stream additions. Finally, `sigmoid_kernel` (2.0%) implements the SwiGLU activation gate in the MLP blocks. At short context lengths these kernels are small in absolute terms, but at ctx=1024 the softmax-related kernels alone collectively rival or exceed individual GEMM kernels in total GPU time.

Problem d

Solution

Going from a forward-only pass to a full training step (forward + backward + optimizer) shifts the composition of GPU work. The backward pass doubles the GEMM count by adding weight-gradient GEMMs (`tn` layout: $\mathrm{d}W = X^\top \mathrm{d}Y$) and input-gradient GEMMs (`nn` layout: $\mathrm{d}X = \mathrm{d}Y\,W$), and also introduces per-activation elementwise gradient kernels (e.g. `sigmoid_backward_kernel`). The AdamW optimizer step is entirely composed of elementwise operations over the parameter tensors (moment updates, bias corrections, weight updates). From the nsys NVTX profile, the optimizer step accounts for approximately 29% of total training step time for the large model at ctx=512 (144 ms out of 499 ms avg per step), adding a substantial purely elementwise phase. Consequently, while the absolute GEMM count roughly doubles from forward to training, the elementwise kernel fraction increases even more, making GEMM less dominant during training than during forward-only inference.

Problem e

Model	ctx	Softmax (ms)	QK matmul (ms)	V matmul (ms)	Softmax / QK
small	128	46.0	38.6	25.7	1.19×
small	256	47.8	43.0	25.9	1.11×
small	512	45.7	41.0	25.7	1.11×
small	1024	48.4	41.7	27.0	1.16×
medium	128	74.5	71.1	51.1	1.05×
medium	256	78.6	74.6	52.8	1.05×
medium	512	78.9	77.2	53.0	1.02×
medium	1024	163.0	122.5	55.4	1.33×
large	128	103.6	102.7	76.9	1.01×
large	256	110.9	113.5	80.4	0.98×
large	512	134.2	123.9	82.9	1.08×
large	1024	496.4	219.1	21.5	2.27×
xl	128	139.9	142.9	106.0	0.98×
xl	256	142.1	149.5	107.8	0.95×
xl	512	462.2	281.9	89.8	1.64×
xl	1024	806.5	446.4	30.9	1.81×
2.7B	128	81.4	78.0	54.8	1.04×
2.7B	256	83.8	83.5	54.9	1.00×
2.7B	512	316.0	189.5	58.6	1.67×
2.7B	1024	638.4	266.4	30.3	2.40×

Table 3: Total NVTX-annotated GPU time for attention sub-operations. Times are summed across all measurement iterations.

Solution

Table 3 shows the total NVTX-annotated GPU time for the softmax, QK matmul, and V matmul sub-operations across all model sizes and context lengths. At short context lengths (128, 256), softmax and QK matmul take comparable time (ratio $\approx 1\times$). However, at longer contexts (512, 1024), softmax becomes significantly more expensive, reaching up to 2.4× slower than the QK matmul for the 2.7B model at ctx=1024. This disparity is far larger than the FLOPs ratio would predict: the QK matmul costs $\sim 2N^2d$ FLOPs versus softmax's $\sim 3N^2$ FLOPs, so for $d = 64$ (the `large` model) the matmul has $\sim 43\times$ more FLOPs — yet softmax is still 2.27× *slower*, a $\sim 100\times$ discrepancy between the FLOPs ratio and the runtime ratio. This is because softmax is entirely memory-bandwidth bound: it must read and write the full $N \times N$ attention matrix in HBM multiple times (row-max, exponentiation, row-sum, normalization), giving it $O(N^2)$ memory traffic that grows quadratically with sequence length. The QK matmul, by contrast, is compute-bound and benefits from Tensor Core acceleration and SRAM data reuse within tiled GEMM, so its cost grows more favorably. The V matmul times at ctx=1024 appear anomalously low for larger models, likely due to GPU kernel overlap or cache reuse from the recently computed softmax output.

Problem 3: Mixed Precision

Problem 3.1: Mixed Precision Accumulation

Problem a

Accumulator dtype	Value dtype	Result
FP32	FP32	10.0001
FP16	FP16	9.9531
FP32	FP16 (implicit upcast)	10.0021
FP32	FP16 (explicit upcast)	10.0021

Table 4: Results of accumulating 0.01×1000 under different dtype configurations. True value is 10.0.

Solution

The pure FP16 accumulation (case 2) is the least accurate, deviating from the true value of 10.0 by $\sim$0.47% (result: 9.9531) due to FP16's limited mantissa precision causing rounding errors that compound across 1000 additions. Keeping the accumulator in FP32 (cases 3 and 4) recovers accuracy regardless of whether the upcast is implicit or explicit, since PyTorch promotes the FP16 operand to FP32 before performing the addition in both cases.

Problem 3.2: Mixed Precision Benchmarking

Problem a

Solution

Under `torch.autocast` with FP16, the model parameters remain in FP32 (autocast does not modify stored parameters), but the inputs to eligible operations such as linear layers are cast to FP16 at runtime. Specifically: the model parameters within the autocast context are **FP32**; the output of `fc1` is **FP16** (linear layers are autocasted); the output of layer norm is **FP32** (layer norm is excluded from autocasting as it requires higher precision); the predicted logits are **FP16** (output of the final linear layer); the loss is **FP32** (reductions such as cross-entropy are kept in FP32); and the gradients are **FP32** (autograd accumulates gradients in FP32).

Problem b

Solution

Layer normalization is sensitive to mixed precision because it involves a variance computation (a sum of squares) and a division, both of which are reductions that can suffer significant numerical error when accumulated in FP16's limited dynamic range, leading to incorrect normalization. With BF16, layer norm still requires higher-precision accumulation internally, but because BF16 has the same dynamic range as FP32 (8 exponent bits vs. FP16's 5), the risk of overflow and underflow during the reduction is greatly reduced. PyTorch's autocast still runs layer norm in FP32 even with BF16, but the stability concern is less severe than with FP16.

Problem c

Model	FP32 Fwd	BF16 Fwd	Speedup	FP32 Bwd	BF16 Bwd	Speedup
small	14.88	13.83	1.08×	41.21	32.81	1.26×
medium	46.44	36.22	1.28×	125.52	90.49	1.39×
large	119.35	87.40	1.37×	287.88	196.63	1.46×
xl	220.72	158.00	1.40×	543.24	364.46	1.49×
2.7B	281.42	200.03	1.41×	663.62	458.54	1.45×

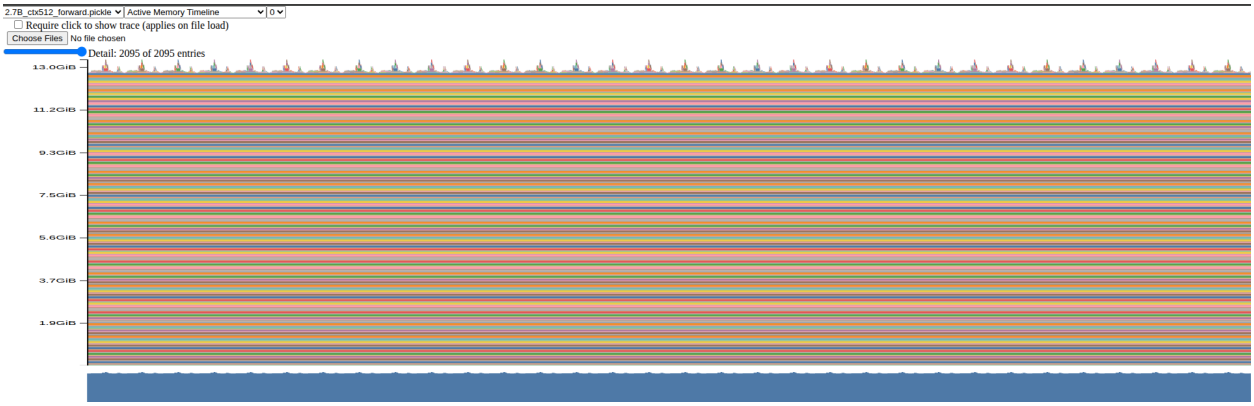
Table 5: Forward and backward pass timings under FP32 and BF16 mixed precision (5 warmup steps, 10 measurement steps, context length 512, batch size 4).

Solution

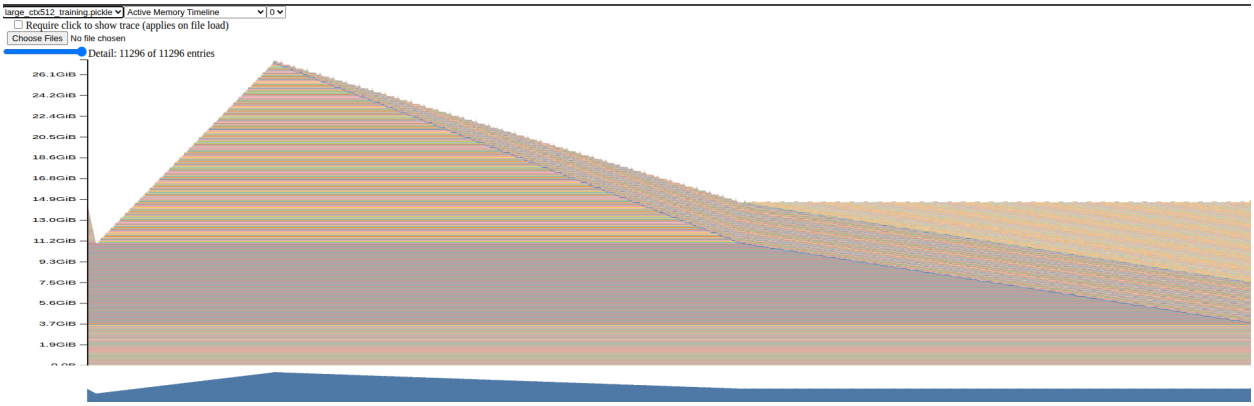
BF16 mixed precision consistently outperforms FP32 across all model sizes, with speedups ranging from 1.08× (small, forward) to 1.49× (xl, backward). The speedup increases with model size and is larger on the backward pass than the forward pass, reflecting that larger models are more compute-bound and therefore benefit more from Tensor Core acceleration at lower precision. The small model shows the least speedup, likely because it has more overhead-dominated operations relative to its GEMM workload.

Problem 4: Memory Profiling

Problem a



(a) 2.7B model, forward pass only (ctx=512)



(b) `large` model, full training step (ctx=512)

Figure 1: Active Memory Timeline from `pytorch.org/memory_viz`.

Solution

The forward-only timeline (Image 1a) is almost entirely flat at ∼13 GiB throughout, since the pass runs under `torch.no_grad()` and only the model parameters occupy memory — no activations are retained. The training step timeline (Image 1b) shows three clearly distinct phases: a steep ramp up to ∼27 GiB as the forward pass runs and activations are retained for backpropagation, followed by a gradual descent as the backward pass consumes and frees activations layer by layer while computing gradients, and finally a lower plateau as the optimizer step updates parameters and frees gradient buffers. Note: the training timeline uses the `large` model as the 2.7B model exceeded GPU memory during training.

Problem b

Context Length	2.7B Fwd Peak (MB)	`large` Training Peak (MB)
128	13,232.80	15,520.76
256	13,351.04	18,587.29
512	13,823.56	28,190.20

Table 6: Peak memory usage during a forward-only pass (2.7B model, FP32) and a full training step (`large` model, FP32). The 2.7B model exceeded GPU memory during the training step and could not be profiled; the `large` model is used in its place for the training column.

Solution

Forward pass peak memory grows modestly with context length (13,233 MB to 13,824 MB for 2.7B), as only activations scale with sequence length while the dominant cost is the parameter memory ($\sim$13 GB for 3.4B parameters at FP32). Training peak memory grows much more steeply with context length (15,521 MB to 28,190 MB for the `large` model), because the backward pass must retain all forward activations including the quadratically-growing attention matrices, and the optimizer step additionally requires two moment buffers per parameter.

Problem c

Solution

BF16 mixed precision forward peak memory for the 2.7B model at ctx=512 is 19,887 MB, which is *higher* than the FP32 forward peak of 13,824 MB. This is counterintuitive but expected: `torch.autocast` keeps both BF16 activations for compute-intensive ops and FP32 copies for precision-sensitive ops (e.g. layer norm), temporarily increasing peak allocation. For training, BF16 peak (24,564 MB for `large` at ctx=512) is lower than FP32 (28,190 MB), as the reduction in activation memory during the backward pass outweighs the overhead from maintaining dual-precision copies.

Problem d

Solution

A residual stream tensor has shape $(\text{batch_size} \times \text{context_length} \times d_{\text{model}}) = (4 \times 512 \times 2560)$, giving 5,242,880 elements. In single-precision (FP32), each element occupies 4 bytes, so the total size is $5{,}242{,}880 \times 4 = 20{,}971{,}520$ bytes $= 20{,}971{,}520/1024^2 \approx \mathbf{20\,MB}$.

Problem e

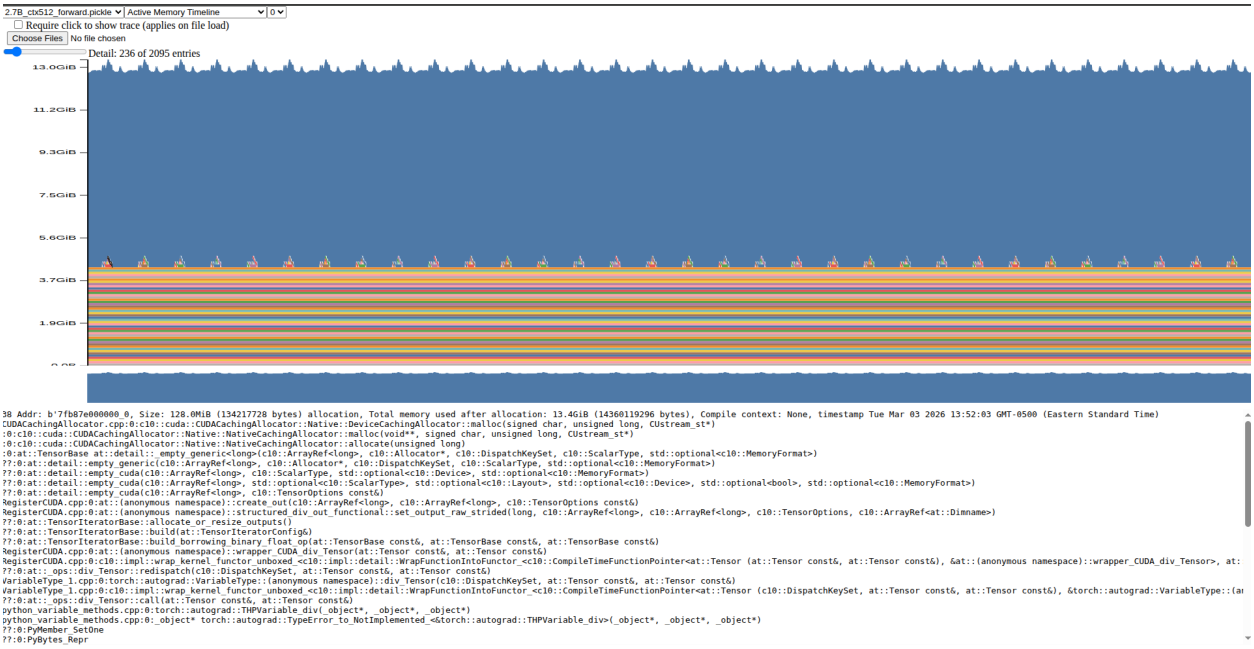


Figure 2: Active Memory Timeline at 10% detail, showing the largest allocations in the 2.7B model forward pass.

Solution

At 10% detail, the largest visible allocations are each **128 MiB**. The stack trace identifies these as the attention score matrix allocated during the scaling division ($QK^\top/\sqrt{d}$) inside `softmax` in `scaled_dot_product_attention` (`nn_utils.py:7`, `model.py:430`). This size can be verified analytically: batch_size $\times$ num_heads $\times$ seq_len^2 $= 4 \times 32 \times 512 \times 512 = 33{,}554{,}432$ elements $\times$ 4 bytes $= 128$ MiB. The repeated peaks visible in the timeline correspond to one such allocation per transformer layer (32 layers total), each freed after its attention computation completes.

Problem 5: Benchmarking PyTorch Attention

d	seq_len	Fwd (ms)	Bwd (ms)	Mem (MB)	Status
16	256	0.076	0.980	21.90	OK
16	1024	0.242	1.338	86.84	OK
16	4096	9.687	33.862	1066.63	OK
16	8192	38.451	133.642	4165.00	OK
16	16384	—	—	—	OOM
32	256	0.076	0.983	23.52	OK
32	1024	0.255	1.434	93.34	OK
32	4096	10.090	33.947	1092.63	OK
32	8192	38.199	133.589	4217.00	OK
32	16384	—	—	—	OOM
64	256	0.076	0.988	26.77	OK
64	1024	0.265	1.401	106.34	OK
64	4096	10.165	34.121	1144.63	OK
64	8192	38.269	134.000	4321.00	OK
64	16384	—	—	—	OOM
128	256	0.076	0.995	33.27	OK
128	1024	0.278	1.456	132.34	OK
128	4096	10.216	34.427	1248.63	OK
128	8192	38.372	134.930	4529.00	OK
128	16384	—	—	—	OOM

Table 7: PyTorch attention forward/backward timings and pre-backward memory usage. Batch size 8, FP32, NVIDIA L40S. Memory is measured before the backward pass.

Solution

Out-of-memory errors occur at all head dimensions for sequence length 16384. We account for the memory usage of the smallest OOM configuration ($d = 16$, seq $= 16384$, batch=8). The dominant cost is the attention score matrix $S = QK^\top$ of shape (batch $\times$ seq $\times$ seq) $= (8 \times 16384 \times 16384)$, requiring $8 \times 16384^2 \times 4$ bytes $= 8{,}589{,}934{,}592$ bytes ≈ 8192 MB. The softmax output P has the same shape, adding another 8192 MB. The Q, K, V, and O tensors are negligible by comparison (~ 8 MB each). Together, these already exceed 16 GB, well beyond the ~ 4165 MB used at seq=8192, which confirms the quadratic memory scaling.

The memory saved for the backward pass (i.e., P, the softmax output retained from the forward pass) grows quadratically with sequence length: $\text{mem}(P) \propto \text{seq}^2$. At seq=1024 this is ~ 32 MB; at seq=8192 it is ~ 2048 MB — a $64\times$ increase in memory for an $8\times$ increase in sequence length. To eliminate this cost, one approach is to not store P at all during the forward pass, instead recomputing it during the backward pass from a compact set of saved checkpoints (Q, K, V, O, and the logsumexp L). This trades additional compute for a significant reduction in memory, bringing peak activation memory from $O(\text{seq}^2)$ down to $O(\text{seq})$.

Problem 6: JIT-Compiled Attention

Problem a

d	seq_len	Uncompiled (ms)			Compiled (ms)			
		Fwd	Bwd	Status	Fwd	Fwd Speedup	Bwd	Bwd Speedup
16	256	0.077	0.965	OK	0.041	1.89×	0.953	1.01×
16	1024	0.242	1.340	OK	0.090	2.69×	0.976	1.37×
16	4096	9.684	33.879	OK	3.366	2.88×	12.903	2.63×
16	8192	38.457	133.740	OK	13.335	2.88×	51.045	2.62×
16	16384	—	—	OOM	53.847	—	203.658	—
32	256	0.076	0.983	OK	0.071	1.08×	1.000	0.98×
32	1024	0.256	1.447	OK	0.105	2.44×	0.997	1.45×
32	4096	10.093	33.943	OK	3.863	2.61×	13.867	2.45×
32	8192	38.192	133.605	OK	19.070	2.00×	59.386	2.25×
32	16384	—	—	OOM	—	—	—	—
64	256	0.076	0.998	OK	0.073	1.04×	0.997	1.00×
64	1024	0.265	1.431	OK	0.114	2.33×	0.977	1.47×
64	4096	10.155	34.109	OK	4.112	2.47×	14.040	2.43×
64	8192	38.262	133.893	OK	19.101	2.00×	59.572	2.25×
64	16384	—	—	OOM	—	—	—	—
128	256	0.077	1.022	OK	0.071	1.08×	1.026	1.00×
128	1024	0.280	1.458	OK	0.137	2.05×	1.039	1.40×
128	4096	10.226	34.426	OK	4.197	2.44×	14.023	2.46×
128	8192	38.379	134.911	OK	19.204	2.00×	60.426	2.23×
128	16384	—	—	OOM	—	—	—	—

Table 8: Compiled vs uncompiled attention forward/backward timings (ms). Batch size 8, FP32, NVIDIA L40S.

Solution

`torch.compile` provides substantial speedups for attention, particularly at longer sequence lengths: forward pass speedups range from ∼1.0–1.9× at short sequences (256) up to ∼2.0–2.9× at longer sequences (4096–8192). Backward pass speedups follow a similar pattern, reaching ∼2.2–2.6× at seq=4096 and seq=8192. Notably, compiled attention with $d = 16$ succeeds at seq=16384 where the uncompiled version OOMs, suggesting the compiler's fused kernel uses less peak memory by avoiding intermediate materializations.

Problem b

Model	Fwd	Cmp Fwd	Speedup	Bwd	Cmp Bwd	Speedup	F+B	Cmp F+B	Speedup
small	14.88	8.06	1.85×	41.21	27.04	1.52×	55.81	36.73	1.52×
medium	46.44	26.69	1.74×	125.52	74.79	1.68×	170.68	106.53	1.60×
large	119.35	71.28	1.67×	287.88	170.86	1.68×	405.77	250.96	1.62×
xl	220.72	134.68	1.64×	543.24	340.42	1.60×	760.15	480.95	1.58×
2.7B	281.42	198.05	1.42×	663.62	485.54	1.37×	941.69	686.56	1.37×

Table 9: Compiled vs uncompiled Transformer timings (ms). Context length 512, batch size 4, FP32, NVIDIA L40S.

Solution

`torch.compile` consistently improves performance across all model sizes, with forward pass speedups ranging from 1.42× (2.7B) to 1.85× (small), and forward+backward speedups of 1.37–1.52×. The smaller models benefit more from compilation as their workloads are more dominated by kernel launch overhead and elementwise ops that the compiler can fuse, while larger models are already more compute-bound with large GEMMs that leave less room for compiler optimisation. For most model sizes, the backward pass sees somewhat less relative speedup than the forward pass, most notably for the small model (1.52× backward vs 1.85× forward), where the backward pass involves more varied gradient operations that are harder for the compiler to fully fuse.

Problem 7: FlashAttention-2 Benchmarking

seq_len	dtype	PyTorch (ms)			FlashAttn (ms)			Speedup	
		Fwd	Bwd	E2E	Fwd	Bwd	E2E	Fwd	E2E
128	bf16	0.024	0.676	0.687	0.008	0.492	0.512	3.00×	1.34×
256	bf16	0.026	0.656	0.703	0.012	0.527	0.527	2.17×	1.33×
512	bf16	0.030	0.661	0.651	0.019	0.546	0.550	1.58×	1.18×
1024	bf16	0.047	0.685	0.696	0.032	0.560	0.564	1.47×	1.23×
2048	bf16	0.092	0.707	0.683	0.058	0.560	0.586	1.59×	1.17×
4096	bf16	0.343	1.103	1.103	0.123	0.581	0.590	2.79×	1.87×
8192	bf16	2.321	8.181	8.182	0.232	2.045	2.046	10.00×	4.00×
16384	bf16	10.153	34.585	34.521	0.750	8.922	8.924	13.54×	3.87×
32768	bf16	40.161	137.245	137.693	2.883	35.093	35.193	13.93×	3.91×
65536	bf16	—	—	—	—	—	—	OOM	
128	fp32	0.027	0.738	0.695	0.008	0.505	0.491	3.38×	1.42×
256	fp32	0.029	0.742	0.708	0.011	0.560	0.559	2.64×	1.27×
512	fp32	0.037	0.676	0.694	0.018	0.572	0.567	2.06×	1.22×
1024	fp32	0.054	0.713	0.694	0.031	0.556	0.579	1.74×	1.20×
2048	fp32	0.119	0.696	0.723	0.057	0.602	0.568	2.09×	1.27×
4096	fp32	0.786	3.148	3.147	0.108	0.935	0.935	7.28×	3.37×
8192	fp32	4.878	16.912	16.897	0.210	3.815	3.820	23.23×	4.42×
16384	fp32	19.176	67.069	66.983	0.581	15.089	15.065	33.01×	4.45×
32768	fp32	76.431	265.754	265.081	2.578	59.812	59.869	29.65×	4.43×
65536	fp32	—	—	—	—	—	—	OOM	

Table 10: FlashAttention-2 Triton vs PyTorch attention timings ($d = 16$, batch=1, causal masking, NVIDIA L40S).

Solution

The Triton FlashAttention-2 implementation significantly outperforms the PyTorch baseline at long sequence lengths, with forward pass speedups reaching up to 33× (FP32, $d = 16$, seq=16384) and 14× (BF16, $d = 16$, seq=32768). Both implementations OOM at seq=65536. The speedup grows with sequence length because the PyTorch implementation materializes the full $O(\text{seq}^2)$ attention matrix in HBM, while the Triton kernel computes attention in tiles and never writes the full matrix to global memory, making its memory traffic $O(\text{seq})$. FP32 consistently shows larger forward speedups than BF16 as the vanilla softmax computation is more bandwidth-bound at higher precision. At larger embedding dimensions ($d = 128$), the forward speedup is reduced or even negative at short sequences, because larger d increases the arithmetic intensity of the tile computation relative to memory traffic, narrowing the gap between tiled and non-tiled implementations.

Backward pass speedups are more modest (∼3–4× at long sequences) than forward pass speedups because the backward pass uses a `torch.compile`d PyTorch implementation with recomputation rather than a custom Triton kernel. This still avoids storing the full attention matrix by recomputing it from saved Q, K, V, O, and L, but runs in standard PyTorch rather than a tiled Triton kernel, limiting the achievable speedup. At short sequence lengths (128–2048), Triton offers modest or no benefit as kernel launch overhead dominates over memory savings.

seq_len	dtype	PyTorch (ms)			FlashAttn (ms)			Speedup	
		Fwd	**Bwd**	**E2E**	**Fwd**	**Bwd**	**E2E**	**Fwd**	**E2E**
128	bf16	0.024	0.694	0.652	0.009	0.542	0.528	2.67×	1.23×
256	bf16	0.026	0.719	0.666	0.014	0.550	0.549	1.86×	1.21×
512	bf16	0.031	0.663	0.669	0.022	0.552	0.557	1.41×	1.20×
1024	bf16	0.049	0.713	0.724	0.038	0.585	0.591	1.29×	1.23×
2048	bf16	0.097	0.694	0.716	0.072	0.555	0.564	1.35×	1.27×
4096	bf16	0.345	1.114	1.114	0.143	0.580	0.603	2.41×	1.85×
8192	bf16	2.323	8.207	8.201	0.305	2.122	2.122	7.62×	3.86×
16384	bf16	9.689	33.647	33.628	1.078	8.214	8.212	8.99×	4.09×
32768	bf16	41.311	133.020	132.974	4.259	32.359	32.383	9.70×	4.11×
65536	bf16	—	—	—	—	—	—	OOM	
128	fp32	0.028	0.691	0.729	0.009	0.536	0.523	3.11×	1.39×
256	fp32	0.030	0.759	0.745	0.012	0.590	0.557	2.50×	1.34×
512	fp32	0.034	0.735	0.706	0.021	0.577	0.579	1.62×	1.22×
1024	fp32	0.058	0.759	0.769	0.036	0.576	0.576	1.61×	1.34×
2048	fp32	0.120	0.694	0.687	0.067	0.561	0.584	1.79×	1.18×
4096	fp32	0.808	3.178	3.179	0.129	0.971	0.971	6.26×	3.27×
8192	fp32	4.878	16.954	16.942	0.254	3.907	3.906	19.20×	4.34×
16384	fp32	19.167	66.929	67.035	0.968	15.155	15.161	19.80×	4.42×
32768	fp32	76.162	265.345	265.541	4.078	61.145	60.790	18.68×	4.37×
65536	fp32	—	—	—	—	—	—	OOM	

Table 11: FlashAttention-2 Triton vs PyTorch attention timings ($d = 32$, batch=1, causal masking, NVIDIA L40S).

seq_len	dtype	PyTorch (ms)			FlashAttn (ms)			Speedup	
		Fwd	**Bwd**	**E2E**	**Fwd**	**Bwd**	**E2E**	**Fwd**	**E2E**
128	bf16	0.025	0.681	0.693	0.011	0.537	0.542	2.27×	1.28×
256	bf16	0.026	0.697	0.687	0.017	0.542	0.523	1.53×	1.31×
512	bf16	0.032	0.658	0.673	0.029	0.578	0.551	1.10×	1.22×
1024	bf16	0.052	0.694	0.683	0.052	0.564	0.565	1.00×	1.21×
2048	bf16	0.107	0.696	0.718	0.099	0.586	0.572	1.08×	1.26×
4096	bf16	0.353	1.124	1.123	0.192	0.583	0.607	1.84×	1.85×
8192	bf16	2.321	8.194	8.194	0.407	2.228	2.227	5.70×	3.68×
16384	bf16	9.719	33.632	33.623	1.446	8.599	8.580	6.72×	3.92×
32768	bf16	40.861	133.466	133.458	5.811	33.788	33.803	7.03×	3.95×
65536	bf16	—	—	—	—	—	—	OOM	
128	fp32	0.029	0.702	0.719	0.011	0.554	0.547	2.64×	1.31×
256	fp32	0.029	0.689	0.687	0.016	0.536	0.570	1.81×	1.21×
512	fp32	0.035	0.684	0.693	0.029	0.573	0.553	1.21×	1.25×
1024	fp32	0.060	0.698	0.700	0.052	0.556	0.566	1.15×	1.24×
2048	fp32	0.144	0.689	0.708	0.098	0.583	0.587	1.47×	1.21×
4096	fp32	0.818	3.216	3.216	0.190	1.062	1.061	4.31×	3.03×
8192	fp32	4.928	17.053	17.066	0.415	4.174	4.177	11.87×	4.09×
16384	fp32	19.434	67.349	67.446	1.426	16.106	16.102	13.63×	4.19×
32768	fp32	76.829	267.284	266.800	5.812	64.031	64.054	13.22×	4.17×
65536	fp32	—	—	—	—	—	—	OOM	

Table 12: FlashAttention-2 Triton vs PyTorch attention timings ($d = 64$, batch=1, causal masking, NVIDIA L40S).

seq_len	dtype	PyTorch (ms)			FlashAttn (ms)			Speedup	
		Fwd	**Bwd**	**E2E**	**Fwd**	**Bwd**	**E2E**	**Fwd**	**E2E**
128	bf16	0.025	0.691	0.692	0.017	0.525	0.526	1.47×	1.32×
256	bf16	0.027	0.653	0.680	0.029	0.514	0.536	0.93×	1.27×
512	bf16	0.035	0.673	0.665	0.050	0.550	0.552	0.70×	1.20×
1024	bf16	0.048	0.699	0.692	0.097	0.576	0.571	0.49×	1.21×
2048	bf16	0.096	0.689	0.699	0.188	0.574	0.559	0.51×	1.25×
4096	bf16	0.338	1.085	1.085	0.384	0.708	0.707	0.88×	1.53×
8192	bf16	2.335	8.247	8.259	0.736	2.525	2.523	3.17×	3.27×
16384	bf16	9.723	33.756	33.791	3.269	10.315	10.278	2.97×	3.29×
32768	bf16	42.257	136.747	136.545	10.048	41.139	41.322	4.21×	3.30×
65536	bf16	—	—	—	—	—	—	OOM	
128	fp32	0.026	0.663	0.674	0.014	0.523	0.520	1.86×	1.30×
256	fp32	0.030	0.673	0.677	0.024	0.565	0.533	1.25×	1.27×
512	fp32	0.038	0.697	0.687	0.041	0.593	0.551	0.93×	1.25×
1024	fp32	0.067	0.689	0.707	0.076	0.561	0.560	0.88×	1.26×
2048	fp32	0.133	0.698	0.705	0.147	0.586	0.572	0.90×	1.23×
4096	fp32	0.880	3.325	3.324	0.289	1.251	1.251	3.04×	2.66×
8192	fp32	5.078	17.397	17.410	0.666	4.744	4.743	7.62×	3.67×
16384	fp32	19.450	67.713	67.705	2.664	17.477	17.500	7.30×	3.87×
32768	fp32	76.813	268.632	268.487	10.918	69.570	69.521	7.04×	3.86×
65536	fp32	—	—	—	—	—	—	OOM	

Table 13: FlashAttention-2 Triton vs PyTorch attention timings ($d = 128$, batch=1, causal masking, NVIDIA L40S).